

THE UNIVERSITY *of* EDINBURGH

Statistical Case Studies

Project 1, part 1: Uncertainty in river flow hydrographs for flood inundation modelling

Prepared by

Anna Scollay (s2480899), Lewis Seath (s2475311), Zacharias Onyejiaka (s2624116)

Word Count: XXXX

October 2026

Contents

1 Executive Summary **2**

2 Data and Exploratory Analysis **2**

2.1 Data Description and Quality 2

2.2 Report Assumptions 3

2.3 Exploratory Analysis 4

2.3.1 Seasonality of High Flows 4

2.3.2 Flood Event Hydrographs 5

2.3.3 Flow Distribution 5

3 Threshold Selection and Independence **6**

3.1 Initial Threshold 6

3.2 Independence of Peaks 6

3.3 Final Threshold 7

4 Uncertainty Analysis **9**

References **11**

Appendix: Code **11**

1 Executive Summary

Current flood models treat the shape of the hydrograph as fixed, varying only the maximum flow. This report explores uncertainty in the maximum flow of a hydrograph peak and the time from the start of the rise to that maximum.

Using ten years of river discharge data from the River Don at Parkhill velocity-area gauging station, we identified 42 independent extreme flow events, across which both quantities varied considerably, with median values of c.104 m^3/s and c.69 hours respectively. Flood events were concentrated in Autumn and Winter, with Autumn events typically showing longer times to peak.

Upon initial investigation, the evidence to suggest a relationship between maximum flow and time to peak was weak. Thus, assuming a fixed hydrograph shape may overlook important uncertainty, and SEPA should consider incorporating variation in time to peak alongside maximum flow in future flood risk modelling guidance.

2 Data and Exploratory Analysis

2.1 Data Description and Quality

Table 1: Data used in this analysis, and the quality codes assigned to each reading

Data description	
Station	Don at Parkhill
Source	SEPA time series data service
Measurement	River discharge (m^3/s)
Catchment area	1,273 km^2
Resolution	15 minutes
Station type	Velocity-area
Period	01 October 2016 – 30 September 2026
Readings	350,592
Missing readings	0
Quality codes	
50 – Good (G)	335,647 readings (95.74%), retained
100 – Estimated (E)	14,915 readings (4.25%), retained
140 – Provisional (Prov)	0 readings (0.00%), excluded
150 – Suspect (S)	0 readings (0.00%), excluded
200 – Unchecked (imported from legacy database) (V)	0 readings (0.00%), excluded
254 – Unchecked (U)	30 readings (0.01%), excluded

The subsequent analysis uses discharge readings (m^3/s) covering 10 water years. Parkhill, the lowest gauging station on the Don, applies a single peak flow rating curve to all measurements.

Each reading is assigned a SEPA quality code (Table 1).

- Estimated ($n = 14,915$): Retained. SEPA defines these as acceptable rating curve extrapolations where the river level exceeds the 3.7 m gauging limit, at which Good readings are capped (Figure 1), so they contain the largest flows and preserve the extremes.
- Unchecked ($n = 30$): Excluded. These are restricted to recent entries (29 September 2026, 16:15 onward) and in sporadic 2017–2018 bursts of at most 45 minutes. Only two coincided with extreme flows, so removing them leaves a fully verified data-set with negligible impact.

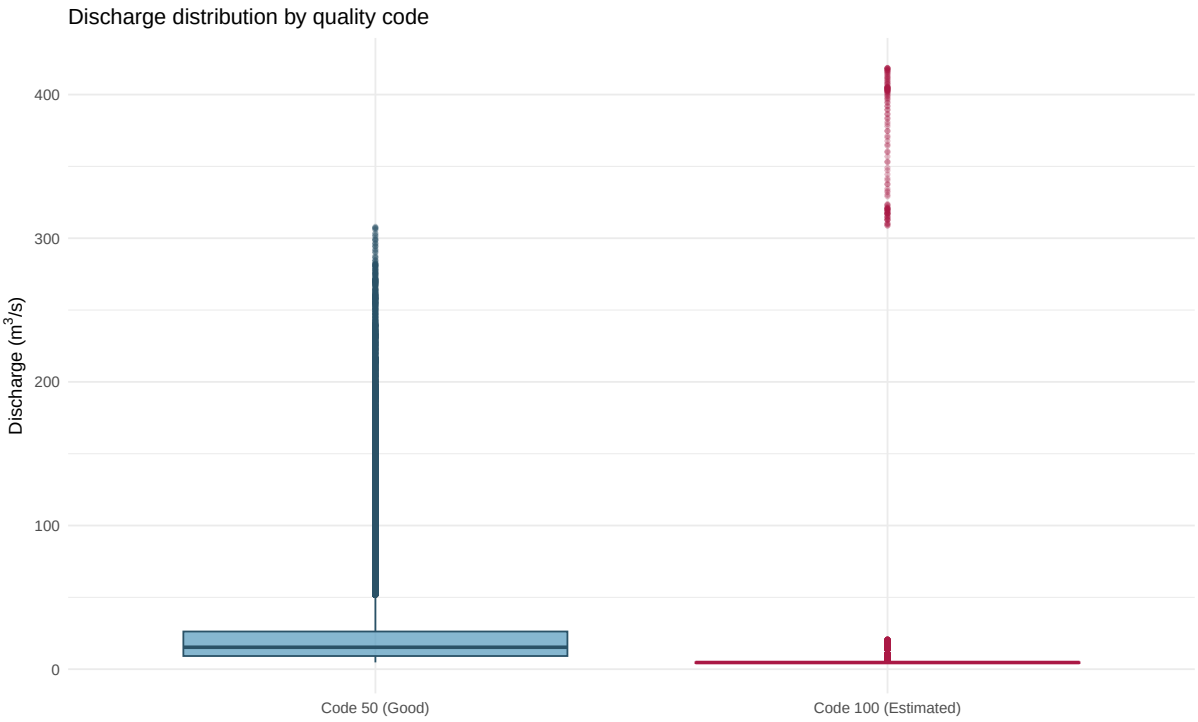


Figure 1: Discharge distribution for quality codes 50 & 100.

2.2 Report Assumptions

- A *clean peak* is the highest candidate discharge within a 14-day symmetrical window, reliably isolating major flood peaks.
- Start of the rise (t_{rise}):

$$t_{\text{rise}} = \max \{t \in [t_{\text{peak}} - 7 \text{ days}, t_{\text{peak}}) \mid q(t) \leq 1.05 \cdot q_{\text{min}}\}$$

where $q_{\text{min}} = \min_{t \in [t_{\text{peak}} - 7 \text{ days}, t_{\text{peak}})} q(t)$ is the lowest discharge recorded in the week before the peak. The 7-day look-back period aligns well with the synoptic cycles of typical flood-generating storms (3-7 days).

2.3 Exploratory Analysis

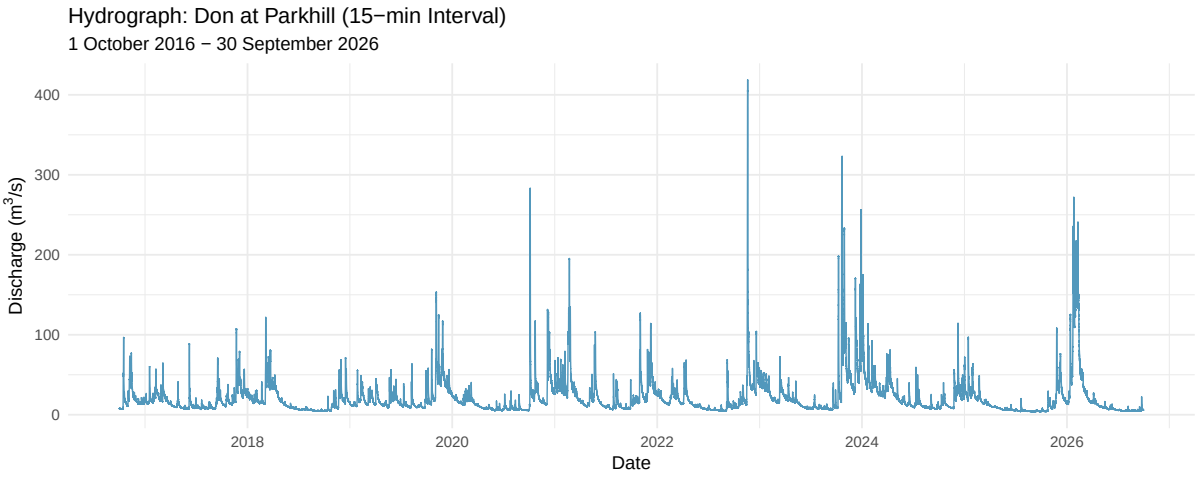


Figure 2: Hydrograph of 15-minute discharge for the Don at Parkhill.

2.3.1 Seasonality of High Flows

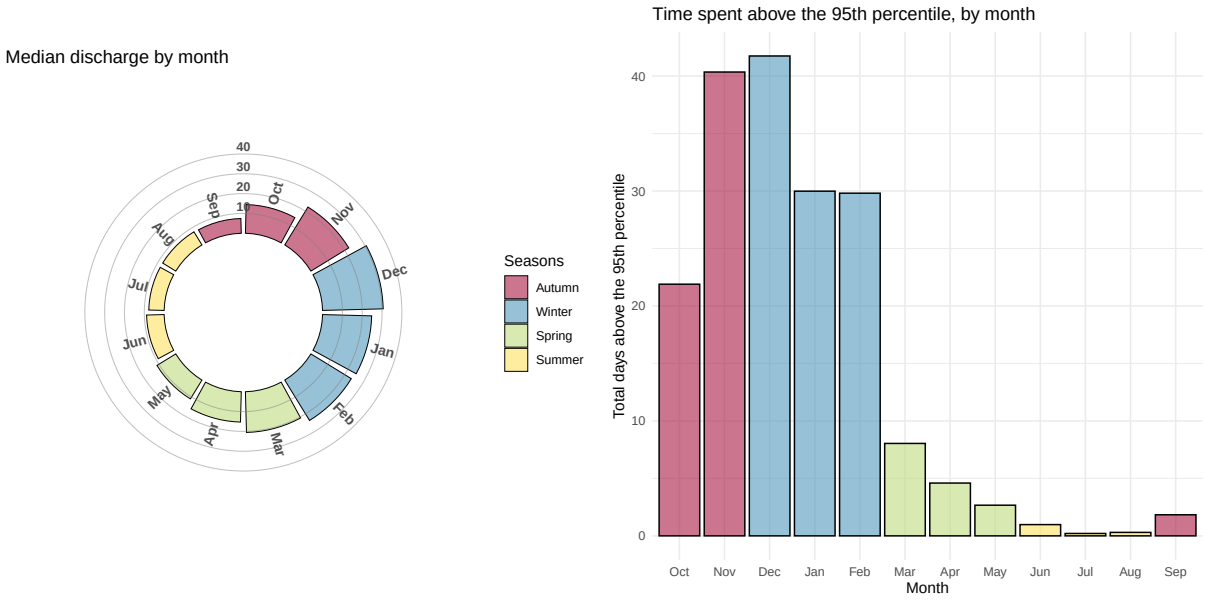


Figure 3: Median discharge by month (left) and total days spent above the 95th percentile in each month (right).

Median discharge is highest from December to February and lowest from July to September. Time spent above the 95th percentile is concentrated between October and February (Figure 3).

2.3.2 Flood Event Hydrographs

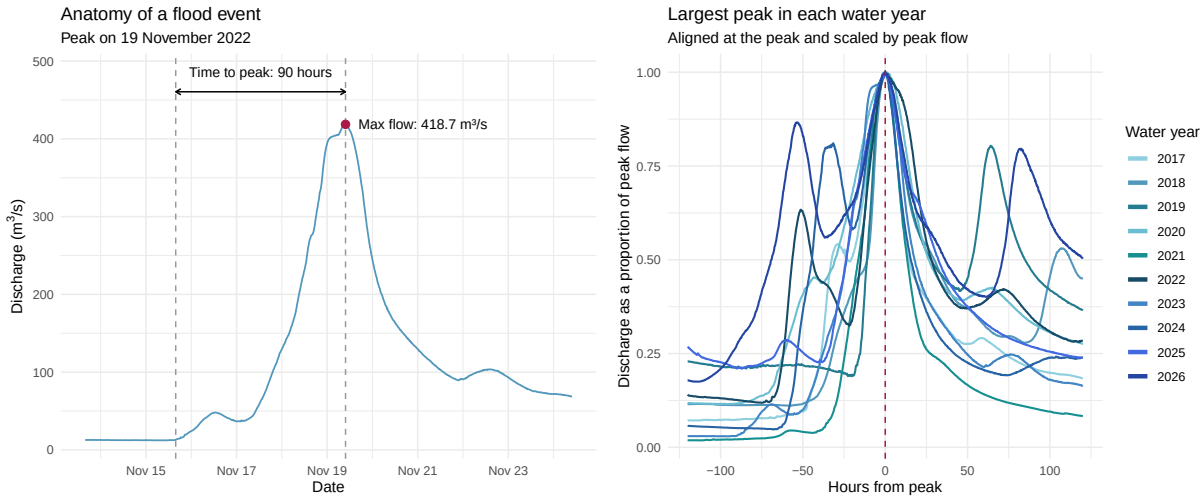
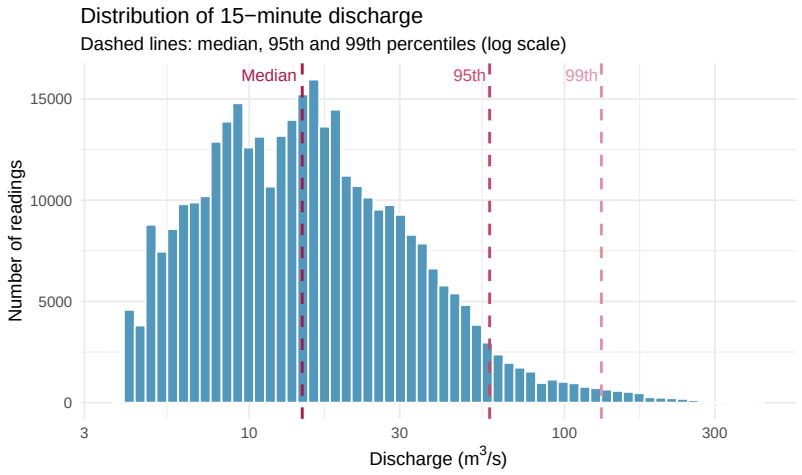


Figure 4: The largest recorded flood event, showing the max flow and time to peak calculated using t_{rise} assumption (left), and the largest peak in each water year, aligned at the peak and scaled by peak flow (right).

Figure 4 shows that hydrograph peak shapes vary considerably, which treating as fixed will overlook, and that smaller peaks often cluster around maxima, so extreme flow events must be filtered for independence.

2.3.3 Flow Distribution



Statistic	Value (m³/s)
Mean	21.65
Standard deviation	24.38
Minimum	3.90
25th percentile	8.74
Median	14.72
75th percentile	25.44
95th percentile	57.88
99th percentile	130.96
Maximum	418.68

Figure 5: Distribution of 15-minute discharge on a log scale, with summary statistics. Dashed lines mark the median, 95th and 99th percentiles.

Discharge is strongly right-skewed (Figure (**?**)(fig:dis15log)), with a maximum over r floor(max(q) / median(q)) times the median, suggesting extreme flows behave differently from typical flows and require separate analysis.

3 Threshold Selection and Independence

3.1 Initial Threshold

We use the Peaks Over Threshold (POT) method to identify extreme flow events exceeding a threshold u, typically set at the 95th percentile or a level yielding 3 to 5 peaks per year. Figure 6 visualises these thresholds: the 95th percentile is 57.9 m^3/s, while raw threshold crossings yield 5, 4, and 3 peaks per year at 79, 93, and 113 m^3/s. However, this raw count overstates distinct events since a single flood can cross the threshold multiple times. Because the 95th percentile yields 9.4 peaks per year, the two rules disagree. We therefore follow the peaks-per-year rule – which directly controls event frequency rather than time above threshold – and set our initial threshold to $u =$ r u_initial m^3/s, giving 5 peaks per year, the upper end of the range (Robson and Reed 1999), since independence filtering will reduce this count.

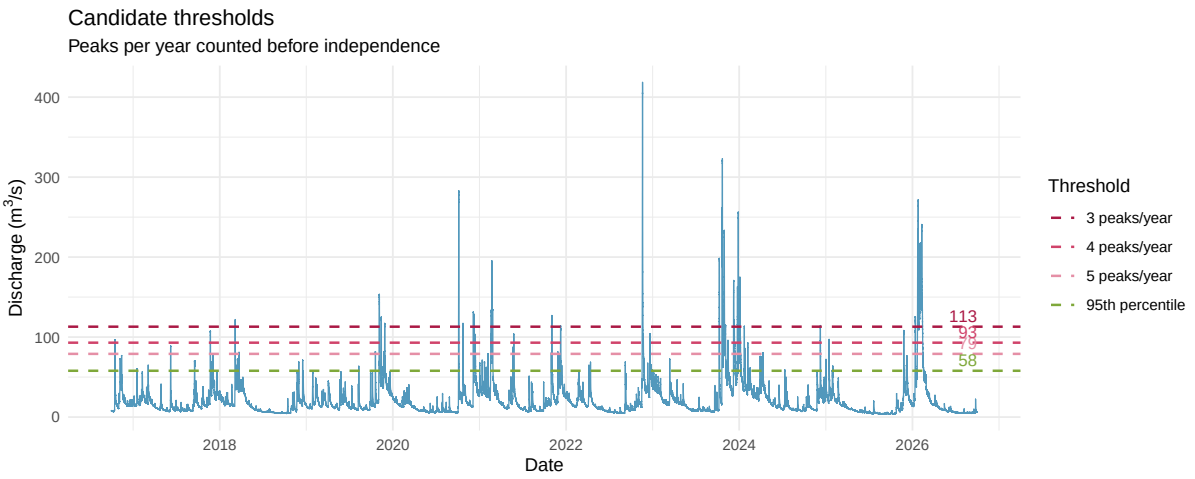


Figure 6: Hydrograph with the candidate thresholds. Peaks per year are counted before applying the independence rules.

3.2 Independence of Peaks

We implement our assumptions to apply the Robson and Reed (1999) independence rules. If either rule fails, we treat the peaks as one event and keep the larger peak.

This reduces the 56 candidate peaks to 32 independent events, or 3.2 per year (Figure 7).

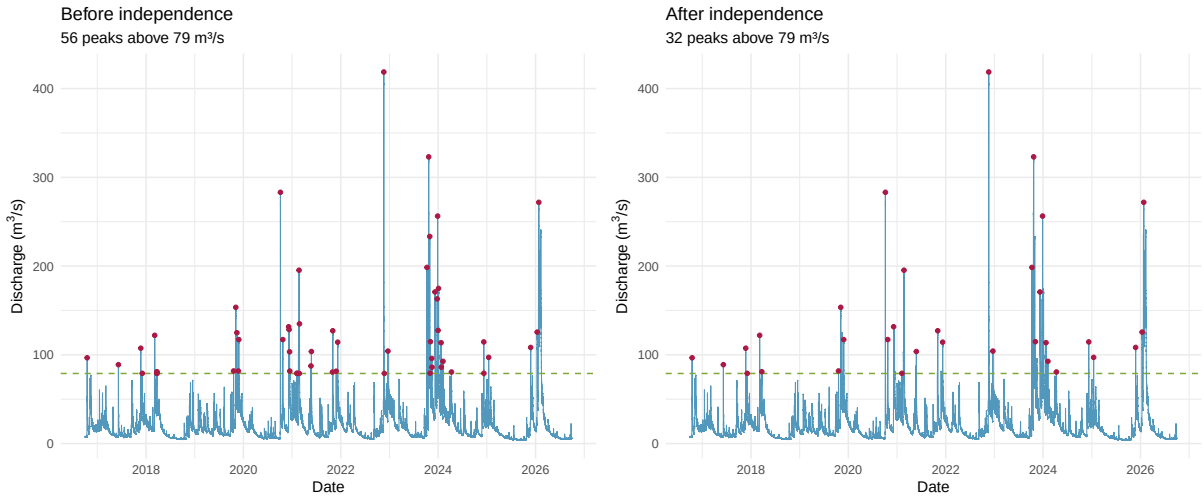


Figure 7: Peaks above the initial threshold before (left) and after (right) applying the independence rules.

3.3 Final Threshold

Our initial threshold yields 3.2 independent peaks per year, whereas thresholds between 57 and 80 m^3/s achieve the target 3 to 5 peaks per year (Figure 8).

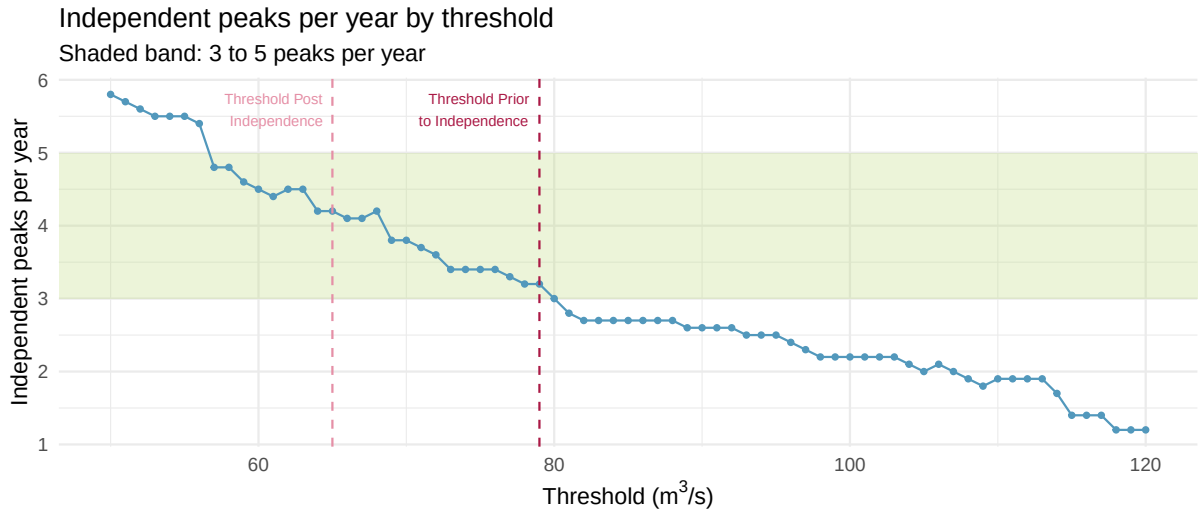


Figure 8: Independent peaks per year at each threshold.

To choose among these, we use the result that excesses over a high enough threshold follow a Generalised Pareto Distribution,

$$P(X - u \leq y \mid X > u) = 1 - \left(1 + \frac{\xi y}{\sigma}\right)^{-1/\xi}, \quad y > 0, \quad 1 + \frac{\xi y}{\sigma} > 0,$$

with scale σ and shape ξ, so that above a valid threshold the mean excess $E[X - u \mid X > u]$ is linear in u and the estimates of ξ and the modified scale, $\sigma - \xi u$, stay roughly constant (Coles 2001). Figure 9 shows that the mean residual life plot is approximately linear and the parameter estimates are roughly constant above 65 m^3/s.

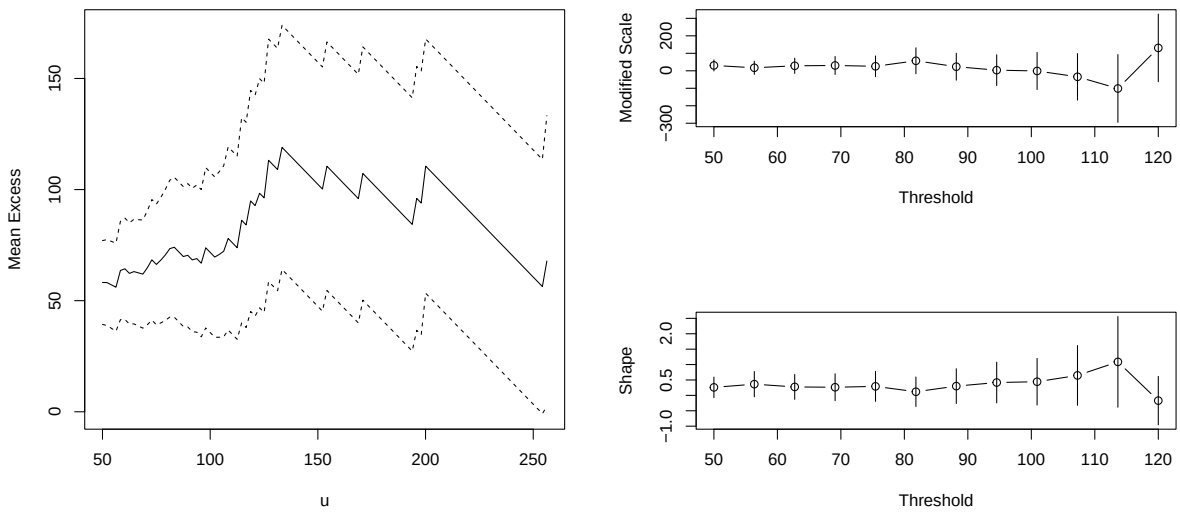


Figure 9: Mean residual life plot (left) and parameter stability plots (right) for independent peaks above 50 m^3/s.

We therefore set $u = ru_final$ m^3/s and, reapplying the independence rules, obtain r nrow(peaks) independent events (4.2 per year), with fitted parameters $\hat{\sigma} = 46.11$ (standard error 11.95) and $\hat{\xi} = 0.278$ (standard error 0.213).

The diagnostic plots (Figure (**?**)(fig:diagnostic)) show a good fit to lower-to-moderate exceedances, but some tail under-estimation and high extrapolation uncertainty at long return periods.

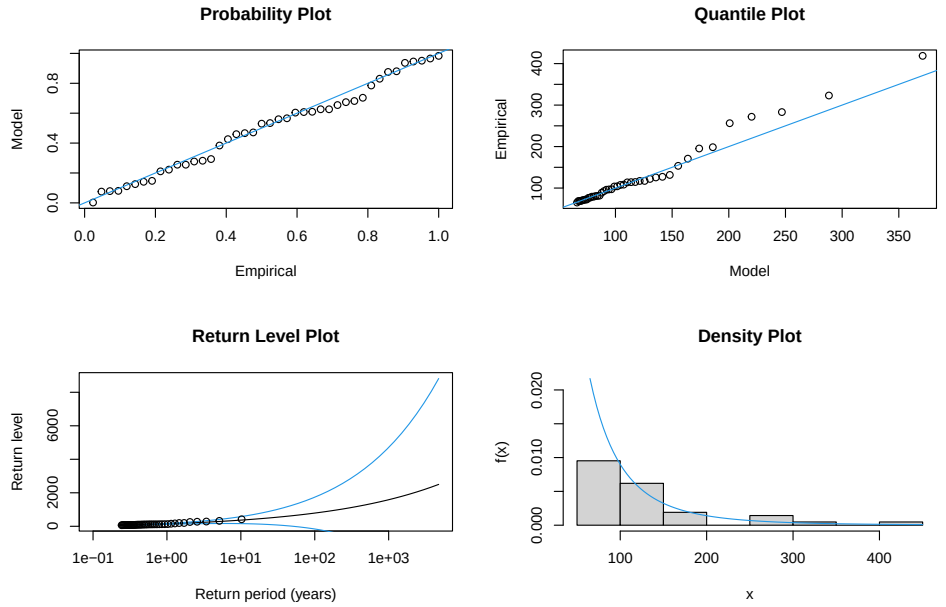


Figure 10: Diagnostic plots for the GPD fitted at the final threshold.

4 Uncertainty Analysis

The final threshold of $u = 65$ m^3/s identifies 44 independent flood events over the ten water years, across which maximum flow and time to peak both varied considerably (Figures 11 and 12). Maximum flow had a median of 103.96 m^3/s (interquartile range 79.2–126.7 m^3/s), while time to peak had a median of 69 hours (interquartile range 36.9–112.4 hours) and ranged from 0 to 166 hours. This suggests that extreme flood hydrographs cannot be adequately characterised by a single peak magnitude or a fixed time to peak.

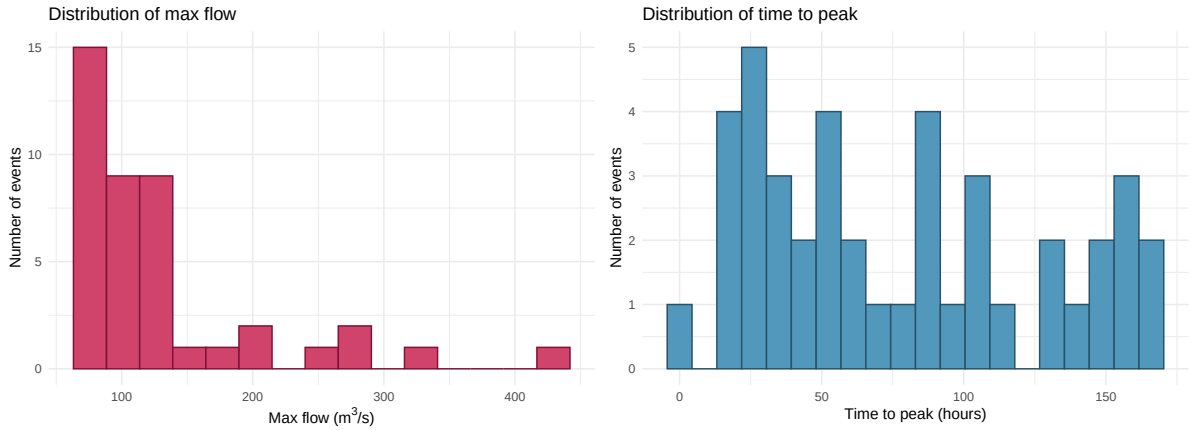


Figure 11: The Distribution of the two parameters

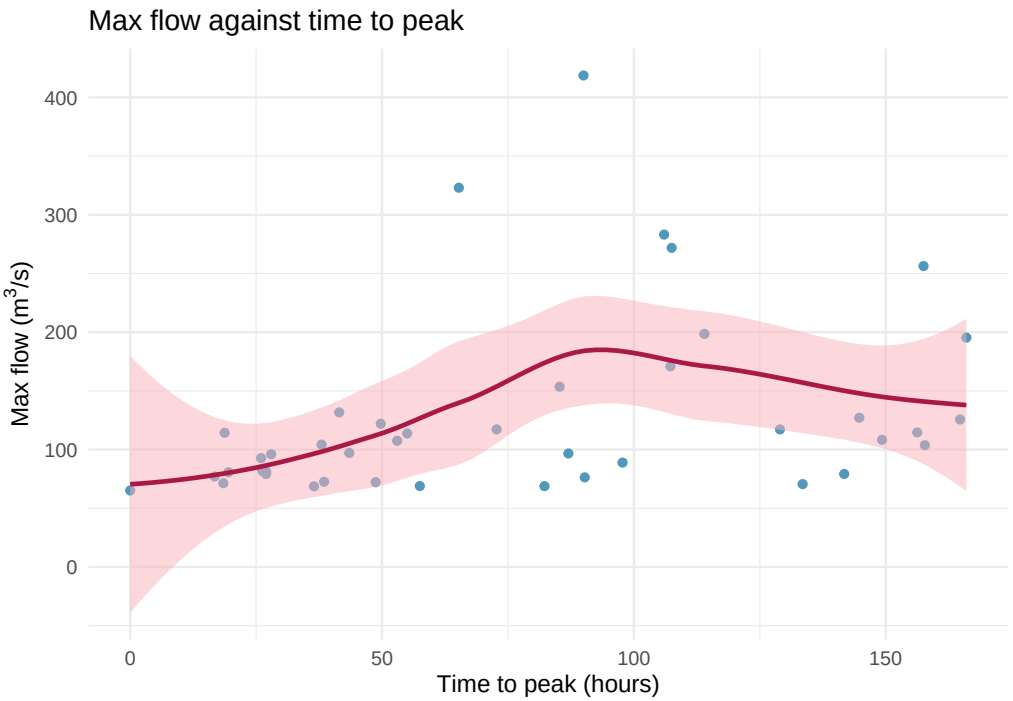


Figure 12: Max flow against time to peak, with a smoothed trend

Since both distributions are skewed and we are interested in any monotonic relationship, we used Spearman's rank correlation, obtaining $\rho = 0.287$ ($p = 0.059$). This indicates a weak positive association, but with insufficient evidence of a monotonic relationship at the 5% significance level, suggesting that larger floods do not necessarily take longer to reach their peak.

Table 2: Spearman's Rank Correlation Between Peak Discharge and Time to Peak

Test	Rho	P-value	S statistic
Spearman's rank correlation	0.496	0.001	6223.756

Table 3: Summary of Max Flow and Time to Peak for each Season

season	Events	Median max flow	Median time to peak (h)
Winter	18	108.951	51.875
Spring	6	80.812	37.500
Summer	1	88.889	97.750
Autumn	17	108.308	85.250

Autumn events (September–November, $n = 17$) had a median maximum flow of 117.19 m^3/s and time to peak of 59 hours, compared with 108.95 m^3/s and 35.5 hours for Winter events (December–February, $n = 18$), suggesting that Autumn floods tend to take longer to peak (Table 3), Figure 13. With so few events per season, however, this difference should be treated with caution, and Spring (6) and Summer (1) had too few events to compare.

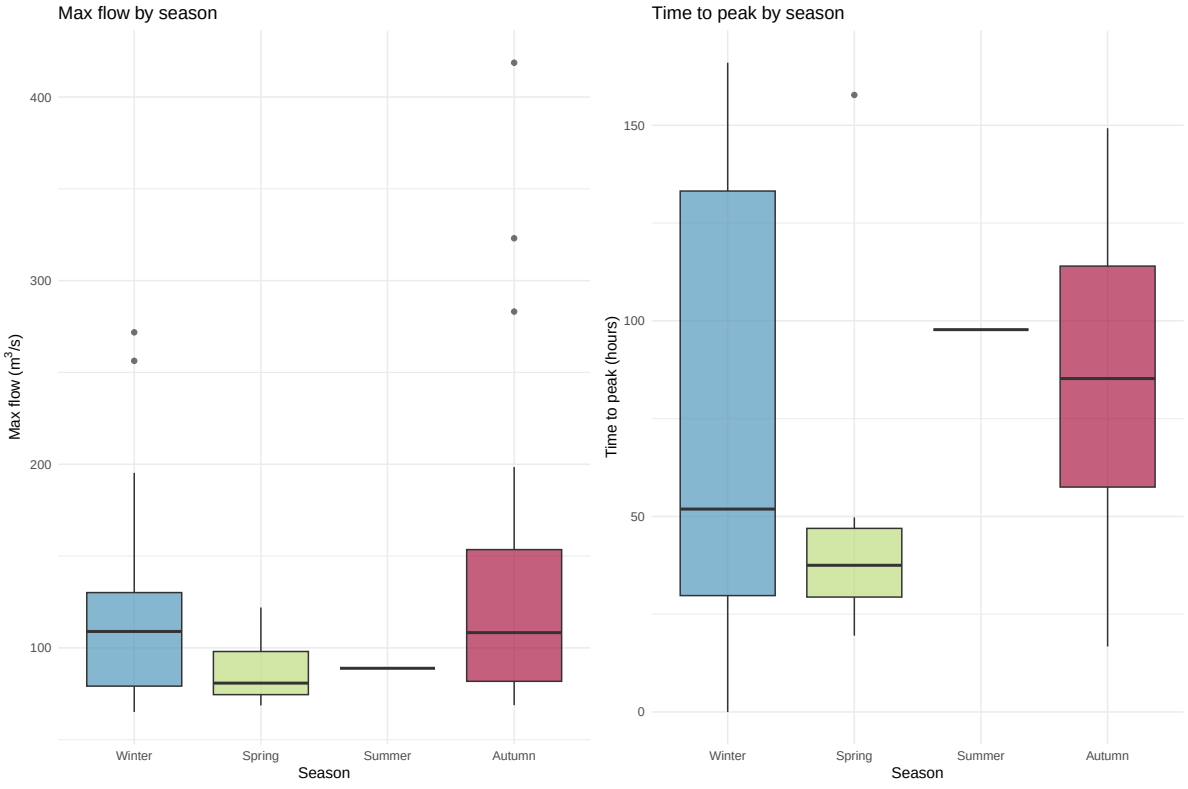


Figure 13: A comparison of the uncertainty in the parameters by season

Repeating the analysis at thresholds of 60-75 m^3/s, median time to peak remained between 43 and 47.25 hours and Spearman's correlation between 0.184 and 0.287 (Table 4), suggesting that the observed variability in time to peak is not simply a consequence of the chosen threshold.

Table 4: Final Uncertainty Table

threshold	events	Events per Year	Median Max Flow	Median Time to Peak (Hours)	Spearman Rho Corelation
55	55	5.5	81.792	65.250	0.308
60	45	4.5	97.149	72.750	0.371
65	42	4.2	103.964	69.000	0.496
70	38	3.8	107.884	79.000	0.462
75	34	3.4	113.975	86.125	0.471

References

Coles, Stuart. 2001. *An Introduction to Statistical Modeling of Extreme Values*. Springer.

Robson, Alice, and Duncan Reed. 1999. *Flood Estimation Handbook, Volume 3: Statistical Procedures for Flood Frequency Estimation*. Institute of Hydrology.

Scottish Environment Protection Agency. 2026. "SEPA Time Series Data Service." https://timeseriesdoc.sepa.org.uk/.

Wilson, A. 2026. "Statistical Case Studies Lecture Slides."

Appendix: Code

```
# ============================================================
# Statistical Case Studies - Project 1 (Part 1)
# Authors: Anna Scollay, Lewis Seath, Zacharias Onyejiaka
# ============================================================

# ------------------------------------------------------------
# Set-up
# ------------------------------------------------------------

# Load packages
library(tidyverse)
library(knitr)
library(kableExtra)
library(ismev)

# ------------------------------------------------------------
# Section 2 - Data and Exploratory Analysis
# ------------------------------------------------------------

# Load data
# ------------------------------------------------------------

# Extend R's default HTTP timeout limit to 300 seconds
options(timeout = 300)

cache_file <- "river_don_parkhill_flow.rds"

if (file.exists(cache_file)) {
  # Load from local cache if file already exists
  river_data <- readRDS(cache_file)
```

```
} else {
  # If no local cache exists, fetch from SEPA in 1-year chunks
  start_years <- seq(2016, 2025, by = 1)

  fetch_chunk <- function(yr) {
    from_date <- paste0(yr, "-10-01T00:00:00")
    to_date   <- paste0(yr + 1, "-09-30T23:59:59")

    url <- paste0(
      "https://timeseries.sepa.org.uk/KiWIS/KiWIS?",
      "service=kisters&type=queryServices&datasource=0&",
      "request=getTimeseriesValues&ts_path=1/234292/Q/15m.Cmd&",
      "from=", from_date, "&to=", to_date, "&",
      "metadata=true&returnfields=Timestamp,Value,Quality%20Code&format=csv"
    )

    read_delim(
      url,
      delim = ";",
      comment = "#",
      col_names = c("Timestamp", "Discharge", "QualityCode"),
      show_col_types = FALSE
    )
  }

  river_data <- map_dfr(start_years, fetch_chunk) %>%
    mutate(Timestamp = as.POSIXct(Timestamp, format = "%Y-%m-%dT%H:%M:%S")) %>%
    filter(!is.na(Discharge)) %>%
    distinct(Timestamp, .keep_all = TRUE)

  # Save to local file so future runs don't need to use the API
  saveRDS(river_data, cache_file)
}

# Subsection 2.1 - Data Description and Quality
# ---------------------------------------------------------------

# Data Description

# Count the number of missing readings by comparing our river data against a
# complete 15-minute sequence
complete_sequence <- seq(min(river_data$Timestamp),
                         max(river_data$Timestamp), by = "15 min")
number_missing <- sum(!complete_sequence %in% river_data$Timestamp)

# Create data description table
data_description_table <- tibble(
  Item = c("Station", "Source", "Measurement", "Catchment area",
           "Resolution", "Station type", "Period", "Readings",
           "Missing readings"),
```

```
  Value = c(
    "Don at Parkhill",
    "SEPA time series data service",
    "River discharge ($m^3/s$)",
    "1,273 $km^2$",
    "15 minutes",
    "Velocity-area",
    paste(format(min(river_data$Timestamp), "%d %B %Y"), "-",
          format(max(river_data$Timestamp), "%d %B %Y")),
    format(nrow(river_data), big.mark = ","),
    format(number_missing, big.mark = ",")
  )
)

# Data Quality

# Create quality code table from SEPA guidance, recording which codes we use
quality_code_table <- tibble(
  `Code number` = c(50, 100, 140, 150, 200, 254),
  `Code name` = c(
    "Good",
    "Estimated",
    "Provisional",
    "Suspect",
    "Unchecked (imported from legacy database)",
    "Unchecked"
  ),
  `Abbreviated code name` = c("G", "E", "Prov", "S", "V", "U"),
  Used =
    c("retained", "retained", "excluded", "excluded", "excluded", "excluded")
)

# Join quality code table with the number of readings for each code
quality_combined <- quality_code_table %>%
  left_join(
    river_data %>% count(QualityCode = as.numeric(QualityCode), name = "Count"),
    by = c("Code number" = "QualityCode")
  ) %>%
  mutate(
    Count = coalesce(Count, 0L),
    Percentage = sprintf("%.2f\\%%", Count / sum(Count) * 100)
  )

# Combine into one table, with one row per quality code below the data description
data_table <- bind_rows(
  data_description_table,
  quality_combined %>%
    transmute(
      Item  = paste0(
        `Code number`, " - ", `Code name`, " (", `Abbreviated code name`, ")"),
      Value = paste0(format(Count, big.mark = ",", trim = TRUE), " readings (",
```

```
                      Percentage, "), ", Used)
    )
)

# Combined table

# Create a table describing the data
# Create a table describing the data
tidy_data_table <- kable(
  data_table, col.names = c("", ""),
  format = "latex",
  booktabs = TRUE,
  escape = FALSE,
  caption = "Data used in this analysis, and the quality codes assigned to each reading",
  label = "tidyingthedata") %>%
  kable_styling(
    latex_options = c("striped", "hold_position"),
    font_size = 9,
    full_width = FALSE,
    position = "left"
  ) %>%
  pack_rows("Data description", 1, 9) %>%
  pack_rows("Quality codes", 10, nrow(data_table))

# Filter for Quality Codes 50 and 100, joining with quality_code_table labels
qc_plot_data <- river_data %>%
  mutate(QualityCode = as.numeric(QualityCode)) %>%
  filter(QualityCode %in% c(50, 100)) %>%
  left_join(
    quality_code_table %>% select(`Code number`, `Code name`),
    by = c("QualityCode" = "Code number")
  ) %>%
  mutate(
    Quality_Label = factor(
      paste0("Code ", QualityCode, " (", `Code name`, ")"),
      levels = c("Code 50 (Good)", "Code 100 (Estimated)")
    )
  )

# Generate discharge by quality code box-plot
qc_plot <- ggplot(qc_plot_data,
                  aes(x = Quality_Label, y = Discharge,
                      fill = Quality_Label, colour = Quality_Label)) +
  geom_boxplot(alpha = 0.7, outlier.size = 1, outlier.alpha = 0.3) +
  scale_fill_manual(values = c(
    "Code 50 (Good)" = "#5298BC",
    "Code 100 (Estimated)" = "#D0436A"
  )) +
  scale_colour_manual(values = c(
    "Code 50 (Good)" = "#2A5267",
    "Code 100 (Estimated)" = "#AA1945"
```

```
  )) +
  labs(
    title = "Discharge distribution by quality code",
    x = NULL,
    y = expression("Discharge (" * m^3 * "/s)")
  ) +
  theme_minimal() +
  theme(
    legend.position = "none"
  )

# Keep only good (50) and estimated (100) readings, matching the Used column above
river_data <- river_data %>%
  filter(as.numeric(QualityCode) %in% c(50, 100))

# Subsection 2.2 - Exploratory Analysis
# ------------------------------------------------------------

# Sort by time and create month, season and water year variables to support our analysis
river_data <- river_data %>%
  arrange(Timestamp) %>%
  mutate(
    month = month(Timestamp, label = TRUE),
    water_year = if_else(month(Timestamp) >= 10,
                         year(Timestamp) + 1, year(Timestamp)),
    season = case_when(
      month(Timestamp) %in% c(12, 1, 2) ~ "Winter",
      month(Timestamp) %in% c(3, 4, 5) ~ "Spring",
      month(Timestamp) %in% c(6, 7, 8) ~ "Summer",
      TRUE ~ "Autumn"
    ),
    season = factor(season, levels = c("Winter", "Spring", "Summer", "Autumn"))
  )

# Shorter names for discharge and time, used in the peak analysis
q <- river_data$Discharge
t <- river_data$Timestamp

# Number of water years
n_years <- n_distinct(river_data$water_year)

# Find where the rise before a peak starts
# Looks back over the previous 7 days (or back to 'earliest' if given) and returns the
# last reading within 5% of the lowest flow in that window
rise_start_row <- function(p, earliest = t[p] - 7 * 24 * 3600) {
  window <- which(t >= earliest & t <= t[p])
  lowest <- min(q[window])
  max(window[q[window] <= 1.05 * lowest])
}

# Months in water year order (October to September)
```

```r
wy_months <- month.abb[c(10:12, 1:9)]

# Colours for each season, used across all seasonal plots
season_colours <- c("Winter" = "#5298BC", "Spring" = "#BFDC80",
                    "Summer" = "#FEE060","Autumn" = "#AA1945")

# High flow definition: the 95th percentile of discharge
high_flow <- quantile(river_data$Discharge, 0.95)

# Hydrograph
hydrograph <- ggplot(river_data, aes(x = Timestamp, y = Discharge)) +
  geom_line(colour = "#5298BC", linewidth = 0.4) +
  labs(
    title = "Hydrograph: Don at Parkhill (15-min Interval)",
    subtitle = "1 October 2016 - 30 September 2026",
    x = "Date",
    y = expression("Discharge ("*m^3*"/s)")
  ) +
  theme_minimal()

# Flow Record and Distribution

# Summary statistics for the 15-minute discharge readings
summary_stats <- river_data %>%
  summarise(
    `Mean`               = mean(Discharge),
    `Standard deviation` = sd(Discharge),
    `Minimum`            = min(Discharge),
    `25th percentile`    = quantile(Discharge, 0.25),
    `Median`             = median(Discharge),
    `75th percentile`    = quantile(Discharge, 0.75),
    `95th percentile`    = quantile(Discharge, 0.95),
    `99th percentile`    = quantile(Discharge, 0.99),
    `Maximum`            = max(Discharge)
  ) %>%
  pivot_longer(everything(), names_to = "Statistic",
               values_to = "Value (m³/s)") %>%
  mutate(`Value (m³/s)` = sprintf("%.2f", `Value (m³/s)`))

# Distrubtion of the data
distribution_colours = c("#AA1945","#D0436A", "#E58FA6")

distribution_plot <- ggplot(river_data, aes(x = Discharge)) +
  geom_histogram(bins = 60, fill = "#5298BC", colour = "white") +
  geom_vline(xintercept = quantile(river_data$Discharge, c(0.5, 0.95, 0.99)),
             linetype = "dashed",
             colour = distribution_colours, linewidth = 0.8) +
  scale_x_log10() +
  labs(
    title = "Distribution of 15-minute discharge",
    subtitle = "Dashed lines: median, 95th and 99th percentiles (log scale)",
```

```
    x = expression("Discharge ("*m^3*"/s)"),
    y = "Number of readings"
  ) +
  theme_minimal()

distribution_plot <- distribution_plot +
  annotate(
    "text",
    x = c(
      median(q, na.rm = TRUE),
      quantile(q, 0.95, na.rm = TRUE),
      quantile(q, 0.99, na.rm = TRUE)
    ),
    y = Inf,
    label = c("Median", "95th", "99th"),
    vjust = 1.5,
    hjust = 1.1,
    size = 3.5,
    colour = distribution_colours
  )

# Seasonality of High Flows

# Median discharge for each month, for the polar plot
monthly <- river_data %>%
  mutate(month_num = as.numeric(format(Timestamp, "%m"))) %>%
  group_by(month_num) %>%
  summarise(value = median(Discharge)) %>%
  mutate(
    month  = month.abb[month_num],
    season = case_when(
      month_num %in% c(9, 10, 11) ~ "Autumn",
      month_num %in% c(12, 1, 2)  ~ "Winter",
      month_num %in% c(3, 4, 5)   ~ "Spring",
      TRUE                        ~ "Summer"
    ),
    season = factor(season, levels = c("Autumn", "Winter", "Spring", "Summer")),
    month_order = (month_num - 10) %% 12   # Oct = 0, Nov = 1, ..., Sep = 11
  ) %>%
  arrange(month_order)
monthly$id <- 1:nrow(monthly)

# Angle and alignment of each month label
monthly$angle <- 90 - 360 * (monthly$id - 0.5) / nrow(monthly)
monthly$hjust <- ifelse(monthly$angle < -90, 1, 0)
monthly$angle <- ifelse(monthly$angle < -90, monthly$angle + 180, monthly$angle)

# Grid lines every 10 up to 40
grid_breaks <- c(10, 20, 30, 40)
y_max <- max(c(monthly$value, 40))
```

```r
# Days spent above the 95th percentile in each month
days_above <- river_data %>%
  filter(Discharge > high_flow) %>%
  mutate(month = factor(month, levels = wy_months)) %>%
  count(month, season) %>%
  mutate(days = n / 96)   # 96 readings per day

#Plot of the distubtion of mean discharges by month
mean_dis_plot <- ggplot(monthly,
                        aes(x = as.factor(id), y = value, fill = season)) +
  geom_blank() +
  geom_col(alpha = 0.6, colour = "black", linewidth = 0.3) +
  geom_hline(yintercept = grid_breaks, colour = "grey50",
             linewidth = 0.3, alpha = 0.5) +
  annotate("text", x = 0.5, y = grid_breaks, label = grid_breaks,
           colour = "grey30", size = 3, fontface = "bold", vjust = -0.3) +
  scale_fill_manual(name = "Seasons", values = season_colours) +
  ylim(-y_max, y_max * 1.4) +
  coord_polar() +
  labs(title = "Median discharge by month") +
  theme_minimal() +
  theme(
    legend.position = "right",
    axis.text = element_blank(),
    axis.title = element_blank(),
    panel.grid = element_blank()
  ) +
  geom_text(aes(x = id, y = value + y_max * 0.05, label = month,
                hjust = hjust, angle = angle),
            colour = "black", alpha = 0.7, size = 3.5, fontface = "bold",
            inherit.aes = FALSE)

# Plot showing time above the 95th percentile by month
time_above_95_plot <- ggplot(days_above,
                             aes(x = month, y = days, fill = season)) +
  geom_col(alpha = 0.6, colour = "black") +
  scale_fill_manual(values = season_colours) +
  labs(
    title = "Time spent above the 95th percentile, by month",
    x = "Month",
    y = "Total days above the 95th percentile"
  ) +
  theme_minimal() +
  theme(legend.position = "none")


# Anatomy of a Flood Event

# The largest recorded peak
flood_peak <- river_data %>% slice_max(Discharge, n = 1, with_ties = FALSE)
peak_time  <- flood_peak$Timestamp
```

```r
peak_flow  <- flood_peak$Discharge

# Start of the rise: using the standardized rise_start_row function
peak_row_single <- match(peak_time, t)
start_row_single <- rise_start_row(peak_row_single)
start_time <- t[start_row_single]

# Time to peak in hours
time_to_peak <- as.numeric(difftime(peak_time, start_time, units = "hours"))

# Readings from two days before the rise to five days after the peak
flood_event <- river_data %>%
  filter(Timestamp >= start_time - 2 * 24 * 3600,
         Timestamp <= peak_time + 5 * 24 * 3600)

#Plot showing the anatomy of a flood year
flood_anatomy_plot <- ggplot(flood_event, aes(x = Timestamp, y = Discharge)) +
  geom_line(colour = "#5298BC", linewidth = 0.6) +
  geom_vline(xintercept = c(start_time, peak_time),
             linetype = "dashed", colour = "gray60") +
  geom_point(data = flood_peak, colour = "#AA1945", size = 2.5) +
  annotate("segment", x = start_time, xend = peak_time,
           y = peak_flow * 1.1, yend = peak_flow * 1.1,
           arrow = arrow(ends = "both", length = unit(0.15, "cm"))) +
  annotate("text",
           x = start_time + (peak_time - start_time) / 2,
           y = peak_flow * 1.16,
           label = paste("Time to peak:",
                         round(time_to_peak), "hours"), size = 3.5) +
  annotate("text", x = peak_time, y = peak_flow, hjust = -0.1,
           label = paste("Max flow:", round(peak_flow, 1), "m³/s"), size = 3.5) +
  labs(
    title = "Anatomy of a flood event",
    subtitle = format(peak_time, "Peak on %d %B %Y"),
    x = "Date",
    y = expression("Discharge ("*m^3*"/s)")
  ) +
  theme_minimal()

# Variation in Flood Hydrograph Shape

# The largest peak in each water year
annual_peaks <- river_data %>%
  group_by(water_year) %>%
  slice_max(Discharge, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  select(water_year, peak_time = Timestamp, peak_flow = Discharge)

# Readings from five days before to five days after each annual peak, with time
# measured from the peak and discharge scaled by the peak flow
annual_peak_shapes <- map_dfr(1:nrow(annual_peaks), function(i) {
```

```r
  peak <- annual_peaks[i, ]
  river_data %>%
    filter(Timestamp >= peak$peak_time - 5 * 24 * 3600,
           Timestamp <= peak$peak_time + 5 * 24 * 3600) %>%
    mutate(
      water_year      = peak$water_year,
      hours_from_peak = as.numeric(difftime(Timestamp, peak$peak_time,
                                            units = "hours")),
      scaled_flow     = Discharge / peak$peak_flow
    )
})

# Plot visualising largest peaks by water year
year_colours <- c(
  "#8ECFE0",
  "#5298BC",
  "#247C91",
  "#6BBED1",
  "#168F91",
  "#174B67",
  "#4285C5",
  "#2764A5",
  "#4169E1",
  "#2445A2",
  "#142D59"
)

peak_wateryear_plot <- ggplot(annual_peak_shapes,
                              aes(x = hours_from_peak, y = scaled_flow,
                                  colour = factor(water_year),
                                  group = water_year)) +
  geom_line(linewidth = 0.7) +
  geom_vline(xintercept = 0, linetype = "dashed", colour = "#AA1945") +
  scale_colour_manual(values = year_colours) +
  labs(
    title = "Largest peak in each water year",
    subtitle = "Aligned at the peak and scaled by peak flow",
    x = "Hours from peak",
    y = "Discharge as a proportion of peak flow",
    colour = "Water year"
  ) +
  guides(colour = guide_legend(
    override.aes = list(linewidth = 1.5)
  ))+
  theme_minimal()
#-------------------------------------------------------------------------------
# Section 3 - Threshold Selection and Independence
#-------------------------------------------------------------------------------

# Candidate peaks: the highest reading in each separate period above threshold u,
# returned as row numbers of river_data
```

```r
candidate_peaks <- function(u) {
  river_data %>%
    mutate(row = row_number(),
           above = Discharge > u,
           run = cumsum(above & !lag(above, default = FALSE))) %>%
    filter(above) %>%
    group_by(run) %>%
    slice_max(Discharge, n = 1, with_ties = FALSE) %>%
    ungroup() %>%
    pull(row)
}


# Subsection 3.1 - Initial Threshold
# ------------------------------------------------------------

# Candidate peaks per year at every whole-number threshold, before independence
crossing_thresholds <- 0:floor(max(q))
crossings_per_year <- sapply(crossing_thresholds, function(u) {
  above <- q > u
  sum(above & !lag(above, default = FALSE))
}) / n_years

# Highest threshold giving at least a target number of candidate peaks per year
crossing_threshold <- function(target) {
  max(crossing_thresholds[crossings_per_year >= target])
}

# The four candidate thresholds
threshold_lines <- tibble(
  Method = c("95th percentile", "5 peaks/year", "4 peaks/year", "3 peaks/year"),
  Value  = c(as.numeric(quantile(q, 0.95)),
             crossing_threshold(5), crossing_threshold(4), crossing_threshold(3))
)

# Initial threshold: the 95th percentile, the lowest of the four
u_initial <- crossing_threshold(5)

# Hydrograph with the four candidate thresholds
threshold_plot <- ggplot(river_data, aes(x = Timestamp, y = Discharge)) +
  geom_line(colour = "#5298BC", linewidth = 0.35) +
  geom_hline(data = threshold_lines, aes(yintercept = Value, colour = Method),
             linetype = "dashed", linewidth = 0.7) +
  geom_text(data = threshold_lines,
            aes(x = max(river_data$Timestamp), y = Value,
                label = round(Value), colour = Method),
            hjust = 1, vjust = -0.4, size = 3.5, show.legend = FALSE) +
  scale_colour_manual(values = c("95th percentile" = "#7FA83A",
                                 "5 peaks/year"    = "#E58FA6",
                                 "4 peaks/year"    = "#D0436A",
                                 "3 peaks/year"    = "#AA1945")) +
```

```
  labs(
    title = "Candidate thresholds",
    subtitle = "Peaks per year counted before independence",
    x = "Date",
    y = expression("Discharge ("*m^3*"/s)"),
    colour = "Threshold"
  ) +
  theme_minimal()


# Subsection 3.2 - Independence of Peaks
# ------------------------------------------------------------

# Apply the Robson and Reed (1999) independence rules to the candidate peaks above u
independent_peaks <- function(u) {

  candidates <- candidate_peaks(u)

  # Clean peaks: absolute maximum of candidate peaks within a +/- 7-day window
  clean <- candidates[sapply(candidates, function(p) {
    window_mask <- abs(as.numeric(difftime(t[candidates],
                                           t[p], units = "days"))) <= 7
    q[p] == max(q[candidates[window_mask]])
  })]
  if (length(clean) < 5) stop("Fewer than five clean peaks above ", u)

  # Average time to rise over the clean peaks, using the full 7-day look-back
  rise_hours <- sapply(clean, function(p) {
    as.numeric(difftime(t[p], t[rise_start_row(p)], units = "hours"))
  })
  min_gap_hours <- 3 * mean(rise_hours)

  # Check each candidate against the last peak kept
  kept <- candidates[1]
  for (p in candidates[-1]) {
    last <- kept[length(kept)]                          # the first peak of the pair
    gap <- as.numeric(difftime(t[p], t[last], units = "hours"))
    trough <- min(q[last:p])                            # flow at the trough between them

    # Rule 1: at least three times the average time to rise apart
    # Rule 2: trough at least two-thirds lower than the first peak
    if (gap >= min_gap_hours & trough <= q[last] / 3) {
      kept <- c(kept, p)                  # independent: a new event
    } else if (q[p] > q[last]) {
      kept[length(kept)] <- p             # same event: keep the larger peak
    }
  }

  # Return the independent peaks, recording the average time to rise
  peaks_u <- river_data[kept, ]
  attr(peaks_u, "mean_rise_hours") <- mean(rise_hours)
```

```r
  peaks_u
}

# Candidate and independent peaks at the initial threshold
candidates_initial <- river_data[candidate_peaks(u_initial), ]
peaks_initial <- independent_peaks(u_initial)
mean_rise_hours <- attr(peaks_initial, "mean_rise_hours")
min_gap_hours <- 3 * mean_rise_hours

# Hydrograph with a set of peaks marked
plot_peaks <- function(peak_data, u, title) {
  ggplot(river_data, aes(x = Timestamp, y = Discharge)) +
    geom_line(colour = "#5298BC", linewidth = 0.35) +
    geom_hline(yintercept = u, linetype = "dashed", colour = "#7FA83A") +
    geom_point(data = peak_data, colour = "#AA1945", size = 1.2) +
    labs(
      title = title,
      subtitle = paste0(nrow(peak_data), " peaks above ", round(u, 1), " m³/s"),
      x = "Date",
      y = expression("Discharge ("*m^3*"/s)")
    ) +
    theme_minimal()
}

# Before and after independence plots, shown side by side in the report
plot_before <- plot_peaks(candidates_initial, u_initial, "Before independence")
plot_after <- plot_peaks(peaks_initial, u_initial, "After independence")


# Subsection 3.3 - Final Threshold
# ------------------------------------------------------------

# Independent peaks per year across a range of thresholds
independent_rates <- tibble(threshold = 50:120) %>%
  mutate(peaks_per_year = sapply(threshold, function(u) {
    peaks_u <- tryCatch(independent_peaks(u), error = function(e) NULL)
    if (is.null(peaks_u)) NA_real_ else nrow(peaks_u) / n_years
  })) %>%
  filter(!is.na(peaks_per_year))

# Thresholds giving 3 to 5 independent peaks per year
thresholds_3to5 <- independent_rates %>%
  filter(peaks_per_year >= 3, peaks_per_year <= 5) %>%
  pull(threshold)

# Independent peaks above a low base threshold, for the GPD diagnostic plots
peaks_base <- independent_peaks(50)

# Final threshold, chosen from the range above using the GPD checks
u_final <- 65
```

```r
# Independent peaks per year against threshold, with the 3-5 band and final threshold
rates_plot <- ggplot(independent_rates, aes(x = threshold, y = peaks_per_year)) +
  annotate("rect", xmin = -Inf, xmax = Inf, ymin = 3, ymax = 5,
           fill = "#BFDC80", alpha = 0.3) +
  geom_line(colour = "#5298BC") +
  geom_point(colour = "#5298BC", size = 1) +
  geom_vline(xintercept = 79, linetype = "dashed", colour = "#AA1945") +
  annotate("text",
    x = 79,
    y = Inf,
    label = "Threshold Prior\nto Independence",
    vjust = 1.5,
    hjust = 1.1,
    size = 2.5,
    colour = "#AA1945"
  ) +
  geom_vline(xintercept = u_final, linetype = "dashed", colour = "#E58FA6") +
  annotate("text",
           x = 65,
           y = Inf,
           label = "Threshold Post\nIndependence",
           vjust = 1.5,
           hjust = 1.1,
           size = 2.5,
           colour = "#E58FA6"
  ) +
  labs(
    title = "Independent peaks per year by threshold",
    subtitle = "Shaded band: 3 to 5 peaks per year",
    x = expression("Threshold ("*m^3*"/s)"),
    y = "Independent peaks per year"
  ) +
  theme_minimal()


# Independent peaks at the final threshold
peaks <- independent_peaks(u_final)

# GPD fitted to the excesses of the independent peak flows above the final threshold
gpd_fit <- gpd.fit(peaks$Discharge, u_final, npy = nrow(peaks) / n_years, show = FALSE)

# Start of each rise: up to 7 days back, but never past the previous peak
# (all peaks are now independent so we do not go further than a previous peak)
peak_rows <- match(peaks$Timestamp, t)
previous_rows <- c(1, head(peak_rows, -1))
start_rows <- mapply(function(p, prev) {
  rise_start_row(p, earliest = max(t[p] - 7 * 24 * 3600, t[prev]))
}, peak_rows, previous_rows)

# The two quantities SEPA asked for
peaks <- peaks %>%
```

```r
  mutate(
    start_time = t[start_rows],
    max_flow = Discharge,
    time_to_peak_h = as.numeric(difftime(Timestamp, start_time, units = "hours"))
  )

#-------------------------------------------------------------------------
# Section 4 - Uncertainty Analysis
#-------------------------------------------------------------------------

# Max flow
Max_flow_plot <- ggplot(peaks, aes(x = max_flow)) +
  geom_histogram(bins = 15, fill = "#D0436A", colour = "#801239") +
  labs(
    title = "Distribution of max flow",
    x = expression("Max flow ("*m^3*"/s)"),
    y = "Number of events"
  ) +
  theme_minimal()

# Time to peak
Time_to_peak_plot <- ggplot(peaks, aes(x = time_to_peak_h)) +
  geom_histogram(bins = 20, fill = "#5298BC", colour = "#2A5267") +
  labs(
    title = "Distribution of time to peak",
    x = "Time to peak (hours)",
    y = "Number of events"
  ) +
  theme_minimal()

Max_vs_time <- ggplot(peaks, aes(x = time_to_peak_h, y = max_flow)) +
  geom_point(color = "#5298BC") +
  geom_smooth(method = "loess", color = "#AA1945", fill = "#FBB1BA", alpha = 0.5) +
  labs(
    title = "Max flow against time to peak",
    x = "Time to peak (hours)",
    y = expression("Max flow ("*m^3*"/s)")
  ) +
  theme_minimal()

cor_result <- cor.test(peaks$time_to_peak_h, peaks$max_flow, method = "spearman",
                       exact = FALSE)


# Correlation table
cor_table <- tibble(
  Test = "Spearman's rank correlation",
  Rho = unname(cor_result$estimate),
  `P-value` = cor_result$p.value,
  `S statistic` = unname(cor_result$statistic)
)
```

```r
cor_result_table <- knitr::kable(
  cor_table,
  digits = 3,
  booktabs = TRUE,
  caption = "Spearman's Rank Correlation Between Peak Discharge and Time to Peak",
  label = "coranalysistable"
)

# Spread of each quantity: quartiles (25%, median, 75%) and range, so we can
# report how much each quantity varies, not just its typical value
max_flow_q <- quantile(peaks$max_flow, c(0.25, 0.5, 0.75))
ttp_q <- quantile(peaks$time_to_peak_h, c(0.25, 0.5, 0.75))
ttp_range <- range(peaks$time_to_peak_h)

# Analysis of uncertanity
# Number of events and typical values in each season
events_season_table <- peaks %>%
  group_by(season) %>%
  summarise(
    Events = n(),
    `Median max flow` = median(max_flow),
    `Median time to peak (h)` = median(time_to_peak_h)
  )

events_season_table <- knitr::kable(
  events_season_table,
  booktabs = TRUE,
  digits = 3,
  caption = "Summary of Max Flow and Time to Peak for each Season",
  label = "seasontable"
)

# Relationship, coloured by season
Max_vs_time_season <- ggplot(peaks, aes(x = time_to_peak_h, y = max_flow )) +
  geom_point(size = 2, aes(colour = season)) +
  scale_color_manual(values = c("Autumn" = "#AA1945", "Winter" = "#5298BC",
                                "Spring" = "#BFDC80", "Summer" = "#FEE060")) +
  labs(
    title = "Max flow against time to peak, by season",
    x = "Time to peak (hours)",
    y = expression("Max flow ("*m^3*"/s)"),
    color = "Season"
  ) +
  theme_minimal()

# Max flow by season
max_season_plot <- ggplot(peaks, aes(x = season, y = max_flow)) +
  geom_boxplot(fill = season_colours, alpha = 0.7) +
  labs(
    title = "Max flow by season",
```

```
    x = "Season",
    y = expression("Max flow ("*m^3*"/s)")
  ) +
  theme_minimal()

# Time to peak by season
time_season_plot <- peaks %>% group_by(season) %>%
  ggplot(aes(x = season, y = time_to_peak_h)) +
  geom_boxplot(fill = season_colours, alpha = 0.7) +
  labs(
    title = "Time to peak by season",
    x = "Season",
    y = "Time to peak (hours)"
  ) +
  theme_minimal()

# Uncertainty final table
results <- tibble()

for (u in c(55, 60, 65, 70, 75)) {
  peaks_test <- independent_peaks(u)

  # Start of each rise and time to peak using the standardized rise_start_row function
  peak_rows <- match(peaks_test$Timestamp, t)
  previous  <- c(1, head(peak_rows, -1))
  start_rows <- mapply(function(p, prev) {
    rise_start_row(p, earliest = max(t[p] - 7 * 24 * 3600, t[prev]))
  }, peak_rows, previous)

  # Calculate time_to_peak_h locally
  time_to_peak_h <- as.numeric(difftime(peaks_test$Timestamp, t[start_rows], units = "hours'

  # One row of results for this threshold
  results <- bind_rows(results, tibble(
    threshold             = u,
    events                = nrow(peaks_test),
    events_per_year       = nrow(peaks_test) / n_distinct(river_data$water_year),
    median_max_flow       = median(peaks_test$Discharge),
    median_time_to_peak_h = median(time_to_peak_h),
    spearman_rho          = cor(peaks_test$Discharge, time_to_peak_h,
                                method = "spearman")
  ))
}


results_table <- knitr::kable(
  results,
  col.names = c("threshold", "events", "events_per_year" = "Events per Year",
                "median_max_flow" = "Median Max Flow",
                "median_time_to_peak_h" = "Median Time to Peak (Hours)",
                "spearman_rho" = "Spearman Rho Corelation"),
```

```
  booktabs = T,
  digits = 3,
  caption = "Final Uncertanity Table",
  label = "uncertaintytable"
) %>%
  kable_styling(
    latex_options = c("striped", "scale_down", "hold_position"),
    font_size = 9)
```